

Tabla Completa de Diagramas UML (Contenido extraído del material)

Tipo de Diagrama	Propósito Principal	Elementos que lo Conforman (detallado)
Diagrama de Casos de Uso	Muestra las distintas operaciones que se esperan de una aplicación o sistema y cómo se relaciona con su entorno (usuarios u otras aplicaciones). Es una herramienta esencial para la captura de requerimientos y para la planificación y control de un proyecto interactivo.	Actor: entidad externa (persona, otro sistema, dispositivo) que interactúa con el sistema; se ubica fuera del rectángulo del sistema. Caso de uso: representado por una <i>elipse</i>; cada caso de uso es una operación completa desarrollada por los actores y por el sistema en un diálogo. El conjunto de casos de uso representa la totalidad de operaciones del sistema. Relaciones: - Include (<>): indica que un caso de uso reutiliza obligatoriamente la funcionalidad de otro (ej.: rutinas comunes). - Extend (<>): indica que un caso de uso amplía la funcionalidad de otro bajo condiciones específicas (ej.: comportamiento opcional o alternativo). - Asociación: línea que une actores y casos de uso (comunicación). Sistema: representado por un rectángulo que encierra los casos de uso; actores quedan fuera. Notas del material: es útil para capturar requerimientos funcionales y planificar alcances (ej.: diagrama "Actualizar librosFT" muestra casos como InsertarFT, EliminarFT, ModificarFT y relaciones <>).
Diagrama de Clases	Muestra el conjunto de clases y objetos importantes del sistema, junto con las relaciones existentes. Presenta la estructura estática de la información y la visibilidad entre clases.	Clase: constructo que define estructura y comportamiento de instancias (se dibuja como un rectángulo con tres divisiones: nombre, atributos y métodos). Atributo: propiedad de una entidad; sintaxis recomendada: <i>visibilidad nombre: tipo = valor_inicial {propiedades}</i> (visibilidad: + público, # protegido, - privado). Operación / Método: conjunto de operaciones que describen el comportamiento; sintaxis: <i>visibilidad nombre(parámetros): tipo_retorno {propiedades}</i>. Objeto: instancia de una clase, con identidad única, estado (valores de atributos) y comportamiento (operaciones). Asociación: línea que une clases; puede incluir roles (nombres en los extremos) y multiplicidad (cardinalidad: 1, *, 0..1, m..n). Ejemplo en el material: Clase <i>Publicación</i> con atributos como <i>#CódigoP: Cadena[2]</i>, <i>-Ncopias: Entero</i>, <i>+Actor: Cadena[30]</i>, y operaciones <i>-Agregar()</i>, <i>-Consultar()</i>, <i>+Listar()</i>.
Diagrama de Objetos	Muestra instancias concretas (objetos) de las clases y las relaciones entre ellas en un momento particular; útil para ejemplificar y verificar el estado y enlaces en ejecución.	Objeto: instancia de una clase, nombrada como <i>nombreObjeto:NombreClase</i>. Valores de atributos: muestran el estado concreto del objeto en ese instante (por ejemplo, <i>título = 'Algoritmos'</i>). Enlaces: representan asociaciones activas entre objetos (pueden servir para validar multiplicidades y roles declarados en el diagrama de clases). Uso: validación y pruebas de diseño, comprobando cómo quedarían las instancias durante la ejecución.

Tipo de Diagrama	Propósito Principal	Elementos que lo Conforman (detallado)
Diagrama de Actividades	Caso especial de diagrama de estados donde la mayoría de los estados son 'estados de acción'. Modela pasos de un algoritmo o flujo de trabajo, y puede detallar un caso de uso, un objeto o un mensaje.	Estado de acción: representa una tarea o acción interna; se dibuja como rectángulo con bordes redondeados y tiene al menos una transición de salida. Transición: flecha que une dos estados de acción; indica que al completarse la acción se continúa al siguiente estado (puede depender de condiciones). Nodos de decisión: puntos de bifurcación condicional entre flujos. Inicio / Final: nodos que marcan el comienzo y fin del proceso. Propósito: modelar procesos internos y flujos de trabajo sin enfatizar eventos externos; ideal para detallar la lógica de casos de uso y algoritmos.
Diagrama de Secuencia	Describe la interacción entre un conjunto de objetos a lo largo del tiempo; importante para detallar casos de uso y operaciones mostrando los mensajes intercambiados entre objetos.	Línea de vida: línea vertical punteada que representa la existencia temporal de un objeto; encabezado rectangular con <i>nombreObjeto:NombreClase</i>. Mensajes: líneas sólidas dirigidas entre objetos que representan llamadas de operación o envío de señales; pueden ser sincronas o asincronas (según notación). Activación: rectángulo sobre la línea de vida que indica el período en que el objeto está ejecutando una operación. Eje temporal: avanza de arriba hacia abajo, mostrando orden cronológico de mensajes. Ejemplo del material: diagrama de secuencia para el proceso "Actualizar Depósito" mostrando interacciones entre <i>USUARIO AUTORIZADO</i>, <i>DEPÓSITO F.T.</i> y <i>TOTAL_D</i>, con mensajes de actualización y confirmación (OK).
Diagrama de Colaboración (Comunicación)	Representa la interacción entre objetos enfatizando la estructura y las relaciones que permiten la colaboración; útil para mostrar el contexto y los roles de los objetos dentro de una operación.	Objetos participantes: cada objeto involucrado en la colaboración. Enlaces / Relaciones: líneas que conectan objetos, mostrando quién puede comunicarse con quién. Mensajes numerados: cada mensaje tiene un número que indica el orden de ejecución (por ejemplo 1, 1.1, 2...). Contexto y ciclos: pueden mostrarse ciclos en la ejecución y distinguir objetos que son atributos, temporales o de uso local, algo que no siempre se aprecia en el diagrama de secuencia.
Diagrama de Estados	Muestra los diferentes estados por los que pasa un objeto durante su vida en una aplicación, junto con los cambios (transiciones) que permiten pasar de un estado a otro.	Estado: identifica un periodo del objeto con una condición característica o en espera de cierto estímulo. Evento: ocurrencia que puede causar una transición (puede ser una condición booleana, recepción de señal/mensaje, o paso de tiempo). Transición: relación entre estados; puede involucrar múltiples fuentes o destinos en transiciones complejas. Estados inicial / final: marcan inicio y fin del ciclo de vida del objeto. Uso: modelar comportamiento dependiente de estado, como máquinas de estados finitos para entidades con fases claras.

Tipo de Diagrama	Propósito Principal	Elementos que lo Conforman (detallado)
Diagrama de Componentes	Representa las componentes físicas de la aplicación —módulos, librerías o unidades desplegables— y sus relaciones. Se enfoca en la estructura de implementación del software.	Componente: unidad física (módulo, biblioteca, archivo ejecutable) que compone la aplicación. Interfaz: punto de conexión que define servicios ofrecidos/consumidos por un componente (puede representarse con símbolos de puerto o provisto/requerido). Dependencias: relaciones de uso entre componentes que indican acoplamientos o requerimientos. Ejemplo del material: diagrama donde los módulos 'Reservación', 'Listado' y 'Actualizar' se relacionan con una interfaz y forman parte de la arquitectura de 'Agencia de Viajes'.
Diagrama de Despliegue	Representa la visualización de los componentes sobre la estructura física del sistema: cómo los artefactos de software se asignan a nodos de hardware en tiempo de ejecución.	Nodo: dispositivo físico o entorno de ejecución (servidor, PC, dispositivo embebido). Componentes desplegados: artefactos de software ubicados en nodos (por ejemplo, base de datos, servicios, interfaz cliente). Conexiones: enlaces de comunicación entre nodos (redes, protocolos). Ejemplo del material: servidor que aloja la base de datos 'Reservaciones' y cliente 'PC' que ejecuta la aplicación 'Agencia de Viajes'; se incluye la notación de <> y nodos etiquetados.

Notas para comprensión profunda

- Los diagramas estructurales (Clases, Objetos, Componentes, Despliegue) modelan la organización estática del sistema.
- Los diagramas de comportamiento (Casos de Uso, Actividades, Secuencia, Colaboración, Estados) representan la dinámica del sistema.
- UML integra ambas perspectivas para ofrecer una visión completa.
- Favorece la comunicación entre analistas, diseñadores y programadores, reduciendo errores.
- Los diagramas se complementan entre sí para representar desde los requerimientos hasta la implementación.